Boxing is the cost

An `Integer` takes 16 B to hold a 4 B `int`, plus a 4 B reference to it.

1,000,000 elements, compact headers on	Retained	Per element
JDK `ArrayList<Integer>`, presized	20.0 MB	20 B
EC `IntArrayList` (primitive)	4.0 MB	4 B
JDK `HashSet<Integer>`	48.4 MB	48 B
EC `IntHashSet` (primitive)	8.4 MB	8 B
JDK `HashMap<Integer, Integer>`	48.4 MB	48 B
EC `IntIntHashMap` (primitive)	16.8 MB	17 B

Every `HashMap` entry is one more object

`HashMap` and `HashSet` wrap each entry in a `HashMap$Node`.

- A node is 24 B with compact headers, 32 B without.
- `HashSet` is a `HashMap` inside: 32.4 MB before counting the `Integer`s.
- EC `UnifiedSet` keeps the elements in its table: 8.4 MB, 4x less.
- Node plus box: about 48 B of heap per element to hold one `int`.

Growing a collection leaves slack and garbage

A list grown from empty is bigger than a presized one, and it dropped arrays on the way.

- `ArrayList` grows by 1.5x: up to 7% unused capacity in this run.
- Every array it outgrew became garbage; JOL only sees the final list.
- `HashMap` tables double: 46 to 50.5 B per entry, depending on n.
- Presize when the size is known: `new ArrayList<>(n)`, `HashMap.newHashMap(n)`.

Compact headers shrink nodes, not boxes

`-XX:+UseCompactObjectHeaders` saves 4 B per header, but objects round up to 8 B.

Retained size	Headers off	Headers on	Saved
`HashMap<Integer, Integer>`, 1 M entries	56.4 MB	48.4 MB	14%
`HashSet<Integer>`, 1 M elements	56.4 MB	48.4 MB	14%
`ArrayList<Integer>`, 1 M, presized	20.0 MB	20.0 MB	0%
`Stream.filter().map().mapToLong()`	304 B	272 B	11%
one `Integer`	16 B	16 B	0%

A stream is an object graph before it runs

An unevaluated pipeline already holds objects, and every operation adds a stage.

Pipeline on an empty list, compact headers on	JDK `Stream`	EC `LazyIterable`
start: `stream()` / `asLazy()`	80 B	16 B
+ `filter` / `select`	144 B	24 B
+ `map` / `collect`	144 B	24 B
+ `distinct`	128 B	16 B
3 steps: filter, map, mapToLong	272 B	96 B

References

- Java Object Layout (JOL), OpenJDK Code Tools
- `Integer.valueOf(int)` (Java SE 25 API)
- `HashMap` (Java SE 25 API)
- `ArrayList` (Java SE 25 API)
- The `java` command: `-XX:+UseCompactObjectHeaders` (Java SE 25)